

Experiment 1: FIR Filter Implementation

Objective

The objectives of this experiment are:

- To become familiar with fixed-point number representations in digital hardware.
- To understand bit-growth and quantization effects in fixed-point arithmetic.
- To analyze different fixed-point architectural choices for FIR filter implementation.

Background

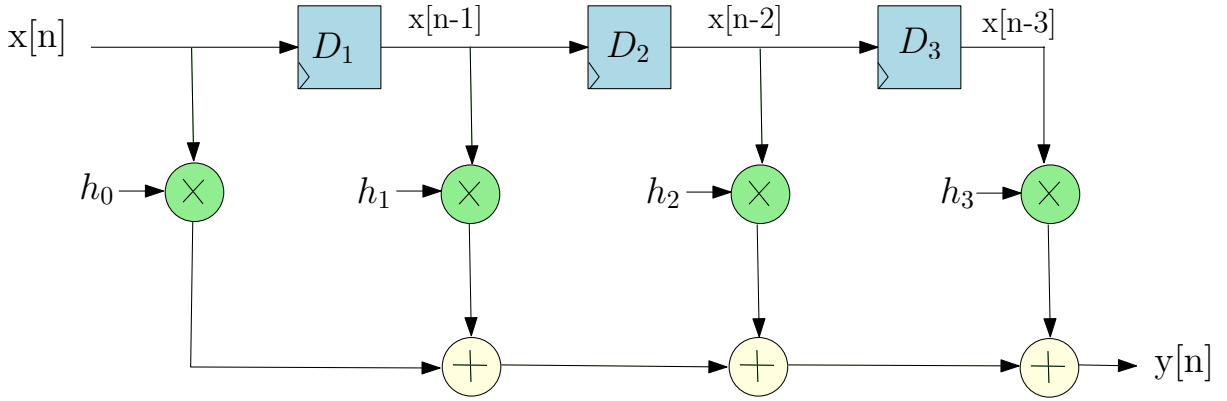


Figure 1: 4-tap FIR Filter

For the 4-tap FIR filter shown in Fig. 1, the output sample is given by:

$$y[n] = h_0 x[n] + h_1 x[n-1] + h_2 x[n-2] + h_3 x[n-3]$$

Experimental Procedure

Part A: Fixed-Point FIR Filter Implementation

- Use Q(1.15) two's complement 16-bit fixed-point representation for FIR filter coefficients, Input and output samples.
- The 4 filter coefficients are $h_0 = 0.0265$, $h_1 = 0.4735$, $h_2 = 0.4735$, and $h_3 = 0.0265$.
- Implement the FIR filter in **Verilog** and verify functionality using a testbench, and use **MATLAB** to generate the golden reference output. To perform the simulation, read the input samples from the file **input.txt** uploaded .

- (d) Compare the Verilog output with MATLAB results and analyze numerical differences.
- (e) Implement and compare the following fixed-point architectures:
 - 1. Rounding after every arithmetic operation.
 - 2. Rounding only after each adder.
 - 3. Rounding only at the final output.
- (f) Use **MATLAB** / **Python** to plot the input and output samples and verify the Verilog simulation output.

Part B: N-tap FIR Filter Design

- (a) Extend the FIR filter design to a higher-order filter using **genvar** and parameterization.
- (b) Verify the design for different filter orders.
- (c) Comment on the impact of filter order on:
 - Bit growth,
 - Output accuracy.